软件设计原则

面向对象设计原则概述

1. Robert C.Martin认为一个可维护性较低的软件设计通常由于四个原因造成：
    1. 过于僵硬 Rigidity
    2. 过于脆弱 Fragility
    3. 复用率低 Immobility
    4. 黏度过高 Viscosity
2. Peter Coad认为一个好的系统设计应该具备
    1. 可扩展性 Extensibility
    2. 灵活性 Flexibility
    3. 可插入性 Pluggability
3. **软件的复用（Reuse）或重用**拥有众多优点，如可以提高软件的开发效率，提高软件质量，节约开发成本，**恰当的复用还可以改善系统的可维护性**
    1. 面向对象设计复用的目标在于**实现支持可维护性的复用**
    2. 面向对象的设计里面，**可维护性复用都是以面向对象设计原则为基础的**
4. **重构（Refactoring）是在不改变软件现有功能的基础上，通过调整程序代码改善软件的质量、性能，使其程序的设计模式和架构更趋合理，提高软件的扩展性和维护性**
5. 一共包括七个原则，他们相互依赖相互补充

UML图

实线箭头（普通箭头）

- 通常表示： 关联 Association / 导航关系 （A 持有/知道 B）
- 读法：箭头指向“被引用/被知道”的一方
- 例： LoginForm -> UserDAO 表示 LoginForm 里有 UserDAO 成员或会调用它

虚线箭头（普通箭头）

- 通常表示： 依赖 Dependency （临时使用）
- 读法：A 依赖 B，通常是方法参数、局部变量、调用了 B 的静态方法等，并不长期持有
    - 例： MainClass ..> LoginForm 可能表示 main 里创建/调用了 LoginForm
      空心三角箭头（白色三角形）

实线 + 空心三角： 泛化/继承 Generalization （extends）

- 例： CircleButton ——▷ AbstractButton

虚线 + 空心三角： 实现 Realization （implements）

- 例： JdbcUserRepository ..▷ UserRepository
  空心三角永远指向： 父类/接口（更抽象的一方）。

菱形（聚合/组合，菱形在“整体”一端）

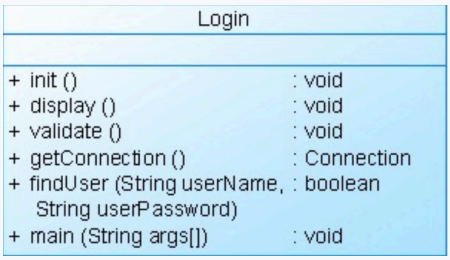
- 空心菱形◇（聚合 Aggregation）：弱“整体-部分”关系，部分可独立存在、可被共享
    - 例： Team ◇—— Member （成员不一定依附某个队伍才能存在）
- 实心菱形◆（组合 Composition）：强“整体-部分”关系，生命周期绑定，整体没了部分也没了

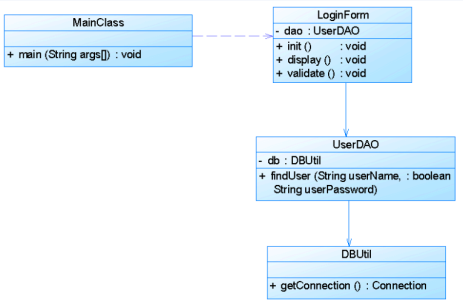
    - 例： Order ◆—— OrderItem （订单没了，订单项也就没有意义）
- 总结
    - 实线箭头 长久占有
    - 虚线箭头 临时借用
    - 实线三角 extends
    - 虚线三角 implements

单一职责原则（Single Responsibility Principle,SRP)

1. 简介：类的职责要单一，不能将太多的职责放在一个类中
2. 定义1：一个对象应该只包含单一的职责（have a single responsibility)，并且该职责被完整地封装在一个类中
3. 定义2：就一个类而言，应该仅有一个引起他变化的原因 There should never be more than one reason for a class to change.
4. 分析：一个类承担的职责越多，他被复用的可能性越小，而且如果一个类承担的职责过多，就相当于将这些职责耦合在一起，当其中一个职责变化时，可能会影响其他职责的运作。 类的职责分成两个方面，数据职责和行为职责，数据职责通过属性体现，行为职责通过方法来体现 单一职责原则则是高内聚，低耦合的指导方针，在很多代码重构手法中都能找到它的存在，它是最简单但又最难运用的原则，需要设计人员发现类的不同职责并将其分离，而发现类的多重职责需要设计人员具有较强的分析设计能力和相关重构经验
5. 实例
    1. 某个基于JAVA的C/S系统的登录功能通过如下登录类实现

       注意：+代表public -代表private





2. 登录验证逻辑validate()需要修改的时候，比如想加一项登录信息验证，需要改变Login类
3. 在数据库查找对应的用户的时候findUser()，需要改变Login类
4. 与数据库取得连接的时候getConnectiona，需要改变Login类
5. 综上所述这个类不是单一职责
6.

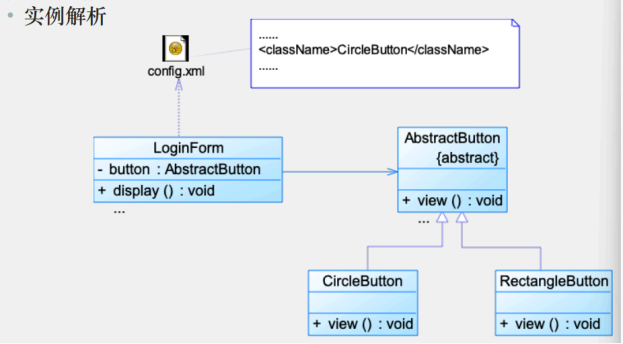
```
class UserService {
    private final UserRepository repo;
    UserService(UserRepository repo) { this.repo = repo; }
    void register(String name) { repo.save(new User(name)); }
}

interface UserRepository { void save(User user); }

class JdbcUserRepository implements UserRepository {
    public void save(User user) { /* 写入数据库 */ }
}

record User(String name) {}
```

开闭原则(Open-Closed Principle, OCP)

1. 定义：一个软件实体应当对扩展开放，对修改关闭。也就是说再设计一个模块的时候，应当是这个模块可以再不被修改的前提下被扩展，即实现再不修改源代码的情况下改变这个模块的行为

   Software entities should be open for extension,but closed for modification.

2. 注意：软件实体可以指一个软件模块、一个由多个类组成的局部结构或者一个独立的类
3. 抽象化是开闭原则的关键
4. 可变性封装原则（Principle of Encapsulation of Variation,EVP)要求找到系统的可变因素，并将其封装起来
5. 实例



总结就是有多个选择的提取一个抽象类封装起来，让上层看不到这个选择的过程，直接就是对上层暴露一个抽象接口，运行时具体确定

```
interface DiscountStrategy { double apply(double price); }

class NoDiscount implements DiscountStrategy {
    public double apply(double price) { return price; }
}

class PercentageDiscount implements DiscountStrategy {
    private final double rate;
    PercentageDiscount(double rate) { this.rate = rate; }
```

```
    public double apply(double price) { return price * (1 - rate); }
}

class PriceCalculator {
    double finalPrice(double price, DiscountStrategy strategy) {
        return strategy.apply(price);
    }
}
```

里氏代换原则(Liskov Substitution Principle, LSP)

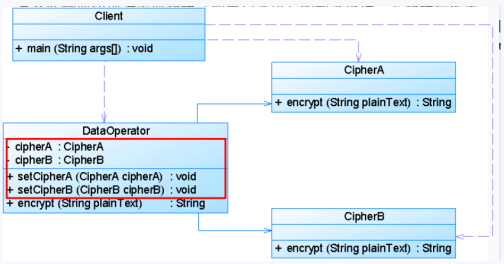
1. 定义1：如果对每一个类型为S的对象o1，都有类型为T的对象o2，使得 以T定义的所有程序P在所有的对象o2都代换成o1时，程序P的 行为没有变化，那么类型S是类型T的子类型。

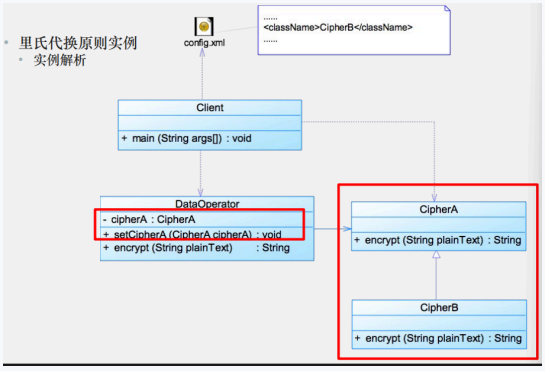
   If for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o1 is substituted for o2 then S is a subtype of T.

2. 定义2：所有引用基类（父类）的地方必须能透明地使用其子类的对象
3. 分析：

   里氏代换原则可以通俗表述为：在软件中如果能够使用基类对象，那么一定能够使用其子类对象。**把基类都替换成它的子类，程序将不会产生任何错误和异常，反过来则不成立**，如果一个软件实体使用的是一个子类的话，那么它不一定能够使用基类。

   里氏代换原则是实现开闭原则的重要方式之一，由于使用 基类对象的地方都可以使用子类对象，因此在程序中尽量 使用基类类型来对对象进行定义，而在运行时再确定其子 类类型，用子类对象来替换父类对象。






```java
interface Bird { void move(); }

class Sparrow implements Bird {
    public void move() { System.out.println("fly"); }
}

class Penguin implements Bird {
    public void move() { System.out.println("swim"); }
}

class BirdShow {
    void show(Bird bird) { bird.move(); }
}
```

依赖倒转原则(Dependence Inversion Principle, DIP)

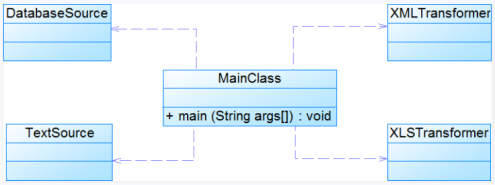
1. 定义1：高层模块不应该依赖低层模块，它们都应该依赖抽象。抽象不应该依赖于细节，细节应该依赖于抽象。

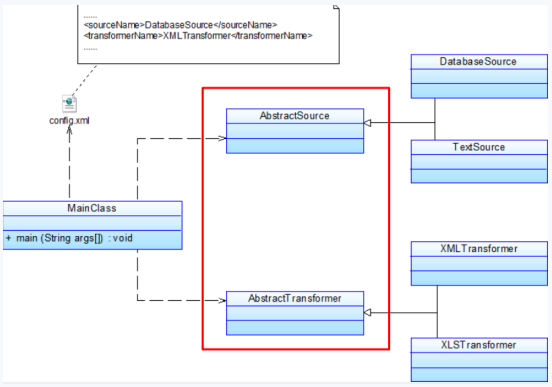
   High level modules should not depend upon low level modules, both should depend upon abstractions. Abstractions should not depend upon details, details should depend upon abstractions.

2. 定义2：要针对接口编程，不要针对实现编程

Program to an interface not an implementation.

3. 简单来说，依赖倒转原则指的是代码要依赖于抽象的类，而不要依赖于具体的类，要针对接口或抽象类编程，而不是针对具体类编程
4. OCP是面向对象设计的目标，DIP是面向对象设计的主要手段
5. 在代码中使用抽象类，而将具体类放在配置文件中，将抽象放进代码，将细节放进元数据
6. 类之间的耦合关系
    1. 零耦合
    2. 具体耦合
    3. 抽象耦合 以抽象方式耦合是依赖倒转原则的关键





高层指调用层直接调用抽象，通过配置决定临时执行的时候用哪个抽象类，高层依赖于中层，底层细节实现依赖于中层，细节实现也是看中层的接口

高层调用/组合/聚合抽象类，低层继承抽象类

```java
interface MessageSender { void send(String to, String text); }

class SmsSender implements MessageSender {
    public void send(String to, String text) { /* 发短信 */ }
}

class NotificationService {
    private final MessageSender sender;
    NotificationService(MessageSender sender) { this.sender = sender; }
    void notify(String user, String text) { sender.send(user, text); }
}
```

接口隔离原则(Interface Segregation Principle, ISP)

1. 定义1：客户端不应该依赖那些它不需要的接口

   Clients should not be forced to depend upon interfaces that they do not use.

2. 定义2：一旦一个接口太大，则需要将它分割成一些更细小的接口，使用该接口的客户端仅需知道与之相关的方法即可

   Once an interface has gotten too 'fat' it needs to be **split into smaller and more specific interfaces** so that any clients of the interface will only know about the methods that pertain to them.

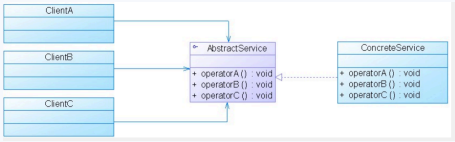
3. 分析：使用多个专门的接口，而不使用单一的总接口。

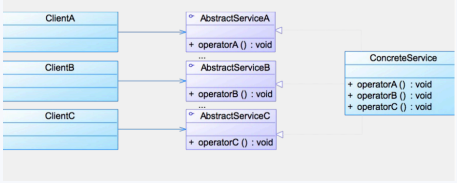
   1. 一个接口就只代表一个角色，每个角色都有它特定 的一个接口，此时这个原则可以叫做“角色隔离原则 ”
   2. 接口仅仅提供客户端需要的行为，即所需的方法， 客户端不需

要的行为则隐藏起来，应当为客户端提供 尽可能小的单独的接口，而不要提供大的总接口。

3. 使用接口隔离原则拆分接口时，首先必须满足**单一职责原则**，将一组相关的操作定义在一个接口 中，且在满足高内聚的前提下，接口中的方法越少越好。

4. 可以在进行系统设计的时候采用定制服务的方式，即为不同的客户短提供宽窄不同的接口，只提供用户需要的行为，而隐藏用户不需要的行为






```java
interface Printer { void print(String content); }
interface Scanner { String scan(); }

class OfficePrinter implements Printer, Scanner {
    public void print(String content) { /* 打印 */ }
    public String scan() { return "doc"; }
}

class SimplePrinter implements Printer {
    public void print(String content) { /* 打印 */ }
}
```

合成复用原则(Composite Reuse Principle, CRP)

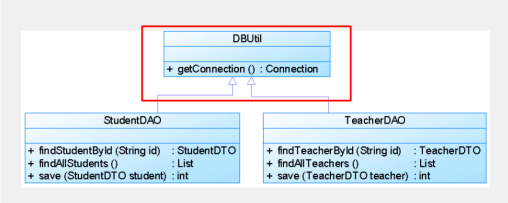
1. 定义：尽量使用对象组合，而不是继承来达到复用的目的。Favor composition of objects over inheritance as a reuse mechanism.

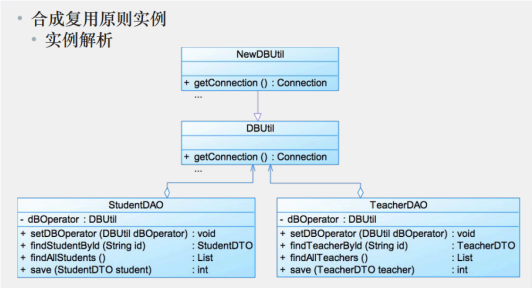
2. 分析：合成复用原则就是指在一个新的对象里通过关联关系（**包括组合关系和聚合关系**）来使用一些已有的对象，使之成为新对象的一部分；新对

象**通过委派调用已有对象的方法达到复用其已有功能的目的。**简言之：**要尽量使用组合/聚合关系，少用继承。**

1. 继承复用：实现简单，易于扩展。破坏系统的 封装性；从基类继承而来的实现是静态的，不 可能在运行时发生改变，没有足够的灵活性； 只能在有限的环境中使用。**“白箱”复用**
2. 组合/聚合复用：耦合度相对较低，选择性地调 用成员对象的操作；可以在运行时动态进行。 **“黑箱”复用**

3. 组合/聚合可以使系统更加灵活，类与类之间的耦 合度降低，一个类的变化对其他类造成的影响相 对较少，因此一般首选使用组合/聚合来实现复用 ；其次才考虑继承，在使用继承时，需要严格遵循里氏代换原则，有效使用继承会有助于对问题的理解，降低复杂度，而滥用继承反而会增加系统构建和维护的难度以及系统的复杂度，因此需要慎重使用继承复用。





连接数据库

1. datasource + pool 连接池
2. drivermanager / 直接建连
3. JPA:实体类、Repository、JPQL/Criteria 来做 CRUD 和查询。

4. 改进解释，前面一张图片中，如果StudentDAO和TeacherDAO连接数据库的方式各不相同，那么就需要重新写一个NewDBUTIL类来负责另外一个数据库的连接，继承的缺点就显露了出来，但是右边并不是自己直接负责连接，我只是使用连接提供方连接好的结果，我把当前的db连接对象注入其中是DBUtil还是NewDBUtil，还是让DBUtil类来负责连接，我调用这个对象的函数，我提供给你我的连接方式，你负责连接就行了，想要换个新的DBUtil，就设置一个NewDBUtil继承自DBUtil，这样就可以在不怎么改动DAO的实现的情况下完成改正了。

```java
interface Engine { void start(); }

class GasEngine implements Engine {
    public void start() { System.out.println("gas"); }
}

class Car {
    private final Engine engine;
    Car(Engine engine) { this.engine = engine; }
    void run() { engine.start(); }
}
```

迪米特法则(Law of Demeter, LoD) 最小知识原则(Least Knowledge Principle, LKP)

(1) 不要和“陌生人”说话。英文定义为：Don't talk to strangers.

(2) 只与你的直接朋友通信。英文定义为：Talk only to your immediate friends.

(3) 每一个软件单位对其他的单位都只有最少的知识，而 且局限于那些与本单位密切相关的软件单位。英文定义 为：Each unit should have only limited knowledge about other units: only units "closely" related to the current unit.

迪米特法则就是指一个软件实体应当 尽可能少的与其他实体发生相互作用。

分析：迪米特法则可分为狭义法则和广义法则。在狭义的迪米特 法则中，如果两个类之间不必彼此直接通信，那么这两个 类就不应当发生直接的相互作用，如果其中的一个类需要 调用另一个类的某一个方法的话，可以通过第三者转发这 个调用。

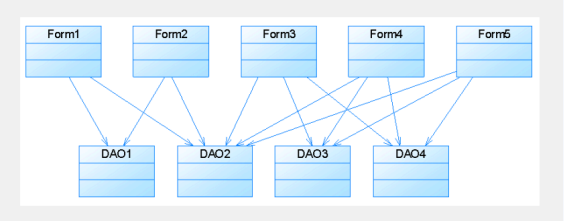
狭义：

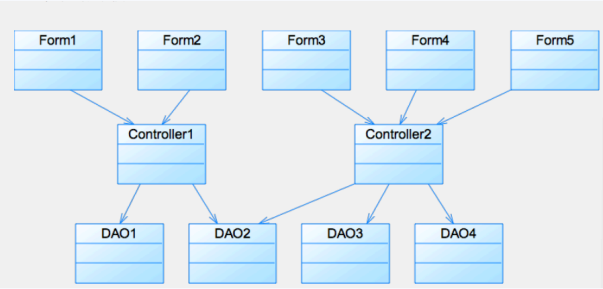
- 不推荐： order.getCustomer().getAddress().getCity()
- 更推荐： order.getCustomerCity() 或 order.getCustomer().getCity() （把

细节封装在更靠近数据的对象里)

广义:

让业务层只依赖 UserRepository 接口，而不是知道 JDBC 连接、SQL、连接池细节






```java
class Order {
    private final Customer customer;
    Order(Customer customer) { this.customer = customer; }
    String customerCity() { return customer.city(); }
}

class Customer {
    private final Address address;
    Customer(Address address) { this.address = address; }
    String city() { return address.city(); }
}

record Address(String city) {}
```

各个原则之间的关系

- 目标：开闭原则
- 指导：最小知识原则
- 基础：单一职责原则、可变性封装原则
- 实现：依赖倒转原则、合成复用原则、里氏代换原则、接口隔离原则

• 依赖倒转原则要求抽象不应该依赖于细节，细节应该 依赖于抽象；要针对接口编程，不要针对实现编程。

• 接口隔离原则要求客户端不应该依赖那些它不需要的 接口，即将一些大的接口细化成一些小的接口供客户 端使用。

• 合成复用原则要求复用时尽量使用对象组合，而不使 用继承。

• 迪米特法则要求一个软件实体应当尽可能少的与其他 实体发生相互作用